

Homework Proteins

Kai Christman (A17522126)

2026-04-20

Original Code

Here is the original code, unedited.

```
# Can you improve this analysis code?
library(bio3d)
s1 <- read.pdb("4AKE") # kinase with drug

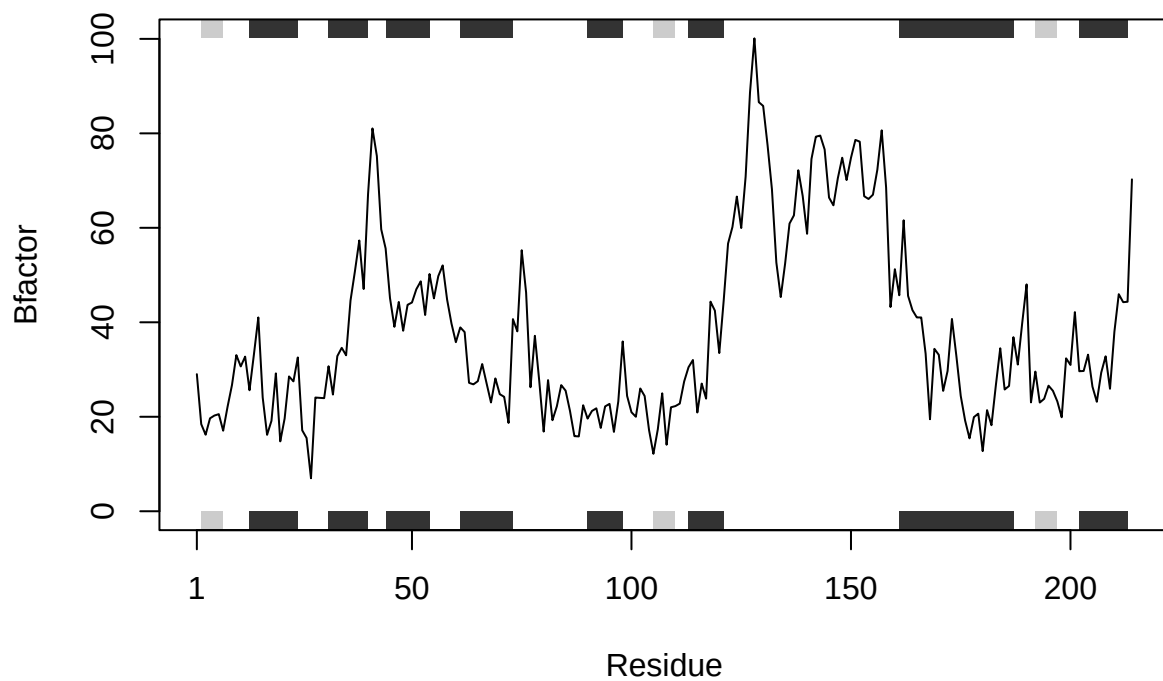
## Note: Accessing on-line PDB file
s2 <- read.pdb("1AKE") # kinase no drug

## Note: Accessing on-line PDB file
## PDB has ALT records, taking A only, rm.alt=TRUE
s3 <- read.pdb("1E4Y") # kinase with drug

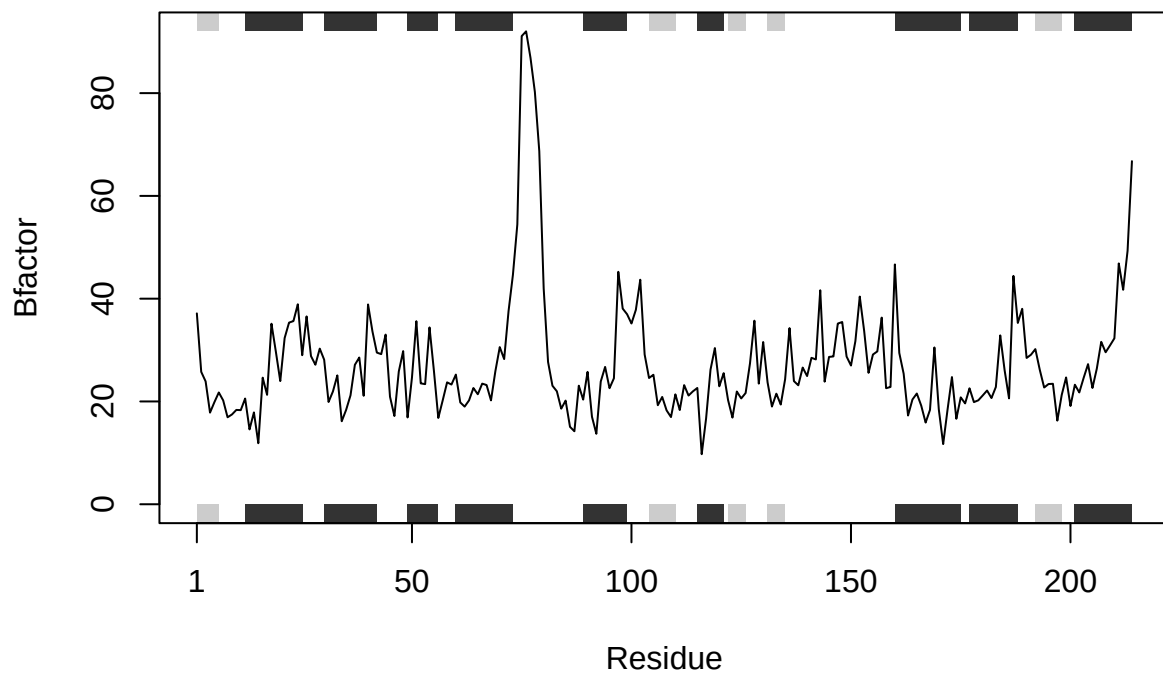
## Note: Accessing on-line PDB file
s1.chainA <- trim.pdb(s1, chain="A", elety="CA")
s2.chainA <- trim.pdb(s2, chain="A", elety="CA")
s3.chainA <- trim.pdb(s1, chain="A", elety="CA")

s1.b <- s1.chainA$atom$b
s2.b <- s2.chainA$atom$b
s3.b <- s3.chainA$atom$b

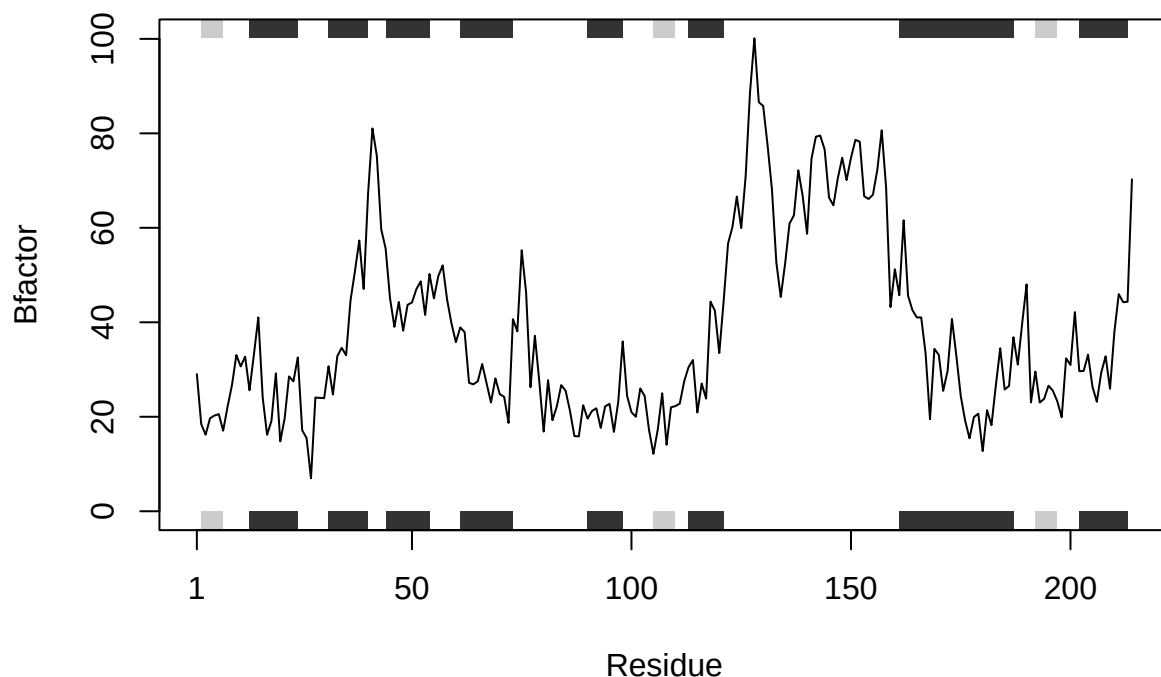
plotb3(s1.b, sse=s1.chainA, typ="l", ylab="Bfactor")
```



```
plotb3(s2.b, sse=s2.chainA, typ="l", ylab="Bfactor")
```



```
plotb3(s3.b, sse=s3.chainA, typ="l", ylab="Bfactor")
```



Protein Data Base Input The previous code is somewhat confusing, so I am going to break it into parts. I want to start with optimization so that the “pdb” is entered with whatever the user desires. In the following segment the protein ID will be input by the user and will be searched for in the PDB.

```
library(bio3d)
load_pdb <- function(pdb_id) {
  pdb <- read.pdb(pdb_id)
  return(pdb)
}
```

The code worked well. The function allows the user to input a certain ID in the protein data bank and then returns relevant information, such as: The total number of models, the total number of atoms in the protein, how many of those atoms belong to the protein, and the amount of side chains.

Editing the Side Chain

The goal now is to allow the user to select the Protein Chain from that protein that they prompted. Many PDB files contain more than a single chain, such as chains A and B. In the previous code, chain A was prompted for all 3 examples, but the following code will allow the user to select a different chain if they chose to. The atom type is also set to “CA” to reflect the alpha carbon.

```
library(bio3d)
load_pdb <- function(pdb_id, chain = "A", atom_type = "CA") {
  pdb <- read.pdb(pdb_id)
  pdb_trim <- trim.pdb(pdb, chain = chain, elety = atom_type)

  return(pdb_trim)
}
```

```

pdb_4ake <- load_pdb("4AKE")

## Note: Accessing on-line PDB file

## Warning in get.pdb(file, path = tempdir(), verbose = FALSE):
## /var/folders/2v/sltcs3bj6w50qp7pg2bk1kl80000gn/T//RtmpG6auwD/4AKE.pdb exists.
## Skipping download

unique(pdb_4ake$atom$chain)

## [1] "A"

unique(pdb_4ake$atom$elety)

## [1] "CA"

```

The user can prompt the “pdb_id”, the “chain” and the “atom_type” which will be filtered through the element types. The `elety` code searches through the different element types and returns the atom of interest, which can be changed to “N” for example if the user wanted to explore Nitrogens rather than Alpha Carbons.

Extract the Beta factors

My edited code only is filtering through a single entry, but I need it to work for multiple Protein ID's. The following code will allow the user to input multiple IDs.

```

library(bio3d)

load_pdb <- function(pdb_ids, chain = "A", atom_type = "CA") {

  results <- vector("list", length(pdb_ids))
  names(results) <- pdb_ids

  for (i in seq_along(pdb_ids)) {
    pdb_id <- pdb_ids[i]

    pdb <- read.pdb(pdb_id)
    pdb_trim <- trim.pdb(pdb, chain = chain, elety = atom_type)

    results[[i]] <- pdb_trim
  }

  return(results)
}

pdb_ids <- c("4AKE", "1AKE", "1E4Y")
pdb_list <- load_pdb(pdb_ids)

## Note: Accessing on-line PDB file

## Warning in get.pdb(file, path = tempdir(), verbose = FALSE):
## /var/folders/2v/sltcs3bj6w50qp7pg2bk1kl80000gn/T//RtmpG6auwD/4AKE.pdb exists.
## Skipping download

## Note: Accessing on-line PDB file

## Warning in get.pdb(file, path = tempdir(), verbose = FALSE):
## /var/folders/2v/sltcs3bj6w50qp7pg2bk1kl80000gn/T//RtmpG6auwD/1AKE.pdb exists.

```

```
## Skipping download

## PDB has ALT records, taking A only, rm.alt=TRUE
## Note: Accessing on-line PDB file

## Warning in get.pdb(file, path = tempdir(), verbose = FALSE):
## /var/folders/2v/sltcs3bj6w50qp7pg2bk1kl80000gn/T//RtmpG6auwD/1E4Y.pdb exists.
## Skipping download
```

The code above uses a `for()` loop to iterate through the inputs the user gives and run the same functions I have previously coded before. It will create a new “**pdb_id**” for each input from the user.

Adding the Plotting and Beta Factors

```
library(bio3d)

load_pdb <- function(pdb_ids, chain = "A", atom_type = "CA") {

  results <- vector("list", length(pdb_ids))
  names(results) <- pdb_ids

  for (i in seq_along(pdb_ids)) {
    pdb_id <- pdb_ids[i]

    pdb <- read.pdb(pdb_id)
    pdb_trim <- trim.pdb(pdb, chain = chain, eley = atom_type)

    b_factors <- pdb_trim$atom$b

    plotb3(
      b_factors,
      sse = pdb_trim,
      typ = "l",
      ylab = "Bfactor",
      main = pdb_id
    )

    results[[i]] <- list(
      pdb = pdb,
      pdb_trim = pdb_trim,
      b_factors = b_factors
    )
  }

  return(results)
}

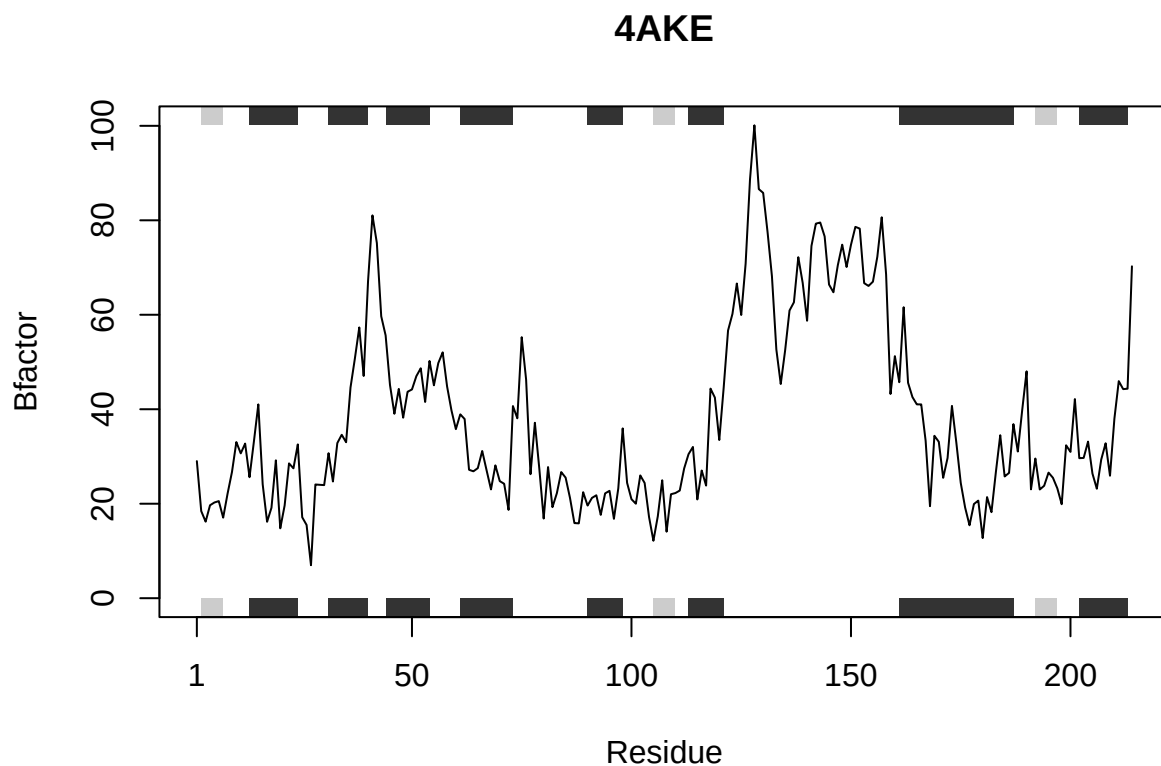
pdb_ids <- c("4AKE", "1AKE", "1E4Y")

results <- load_pdb(pdb_ids)
```

```
## Note: Accessing on-line PDB file

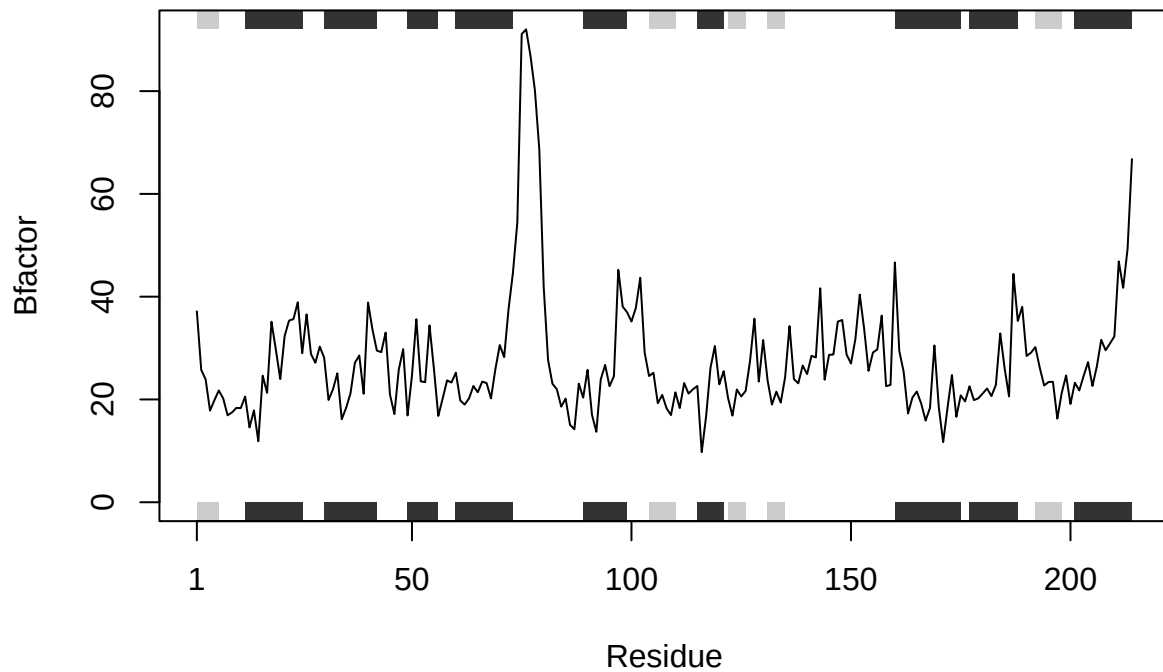
## Warning in get.pdb(file, path = tempdir(), verbose = FALSE):
```

```
## /var/folders/2v/sltcs3bj6w50qp7pg2bk1kl80000gn/T//RtmpG6auwD/4AKE.pdb exists.  
## Skipping download
```



```
## Note: Accessing on-line PDB file  
## Warning in get.pdb(file, path = tempdir(), verbose = FALSE):  
## /var/folders/2v/sltcs3bj6w50qp7pg2bk1kl80000gn/T//RtmpG6auwD/1AKE.pdb exists.  
## Skipping download  
## PDB has ALT records, taking A only, rm.alt=TRUE
```

1AKE

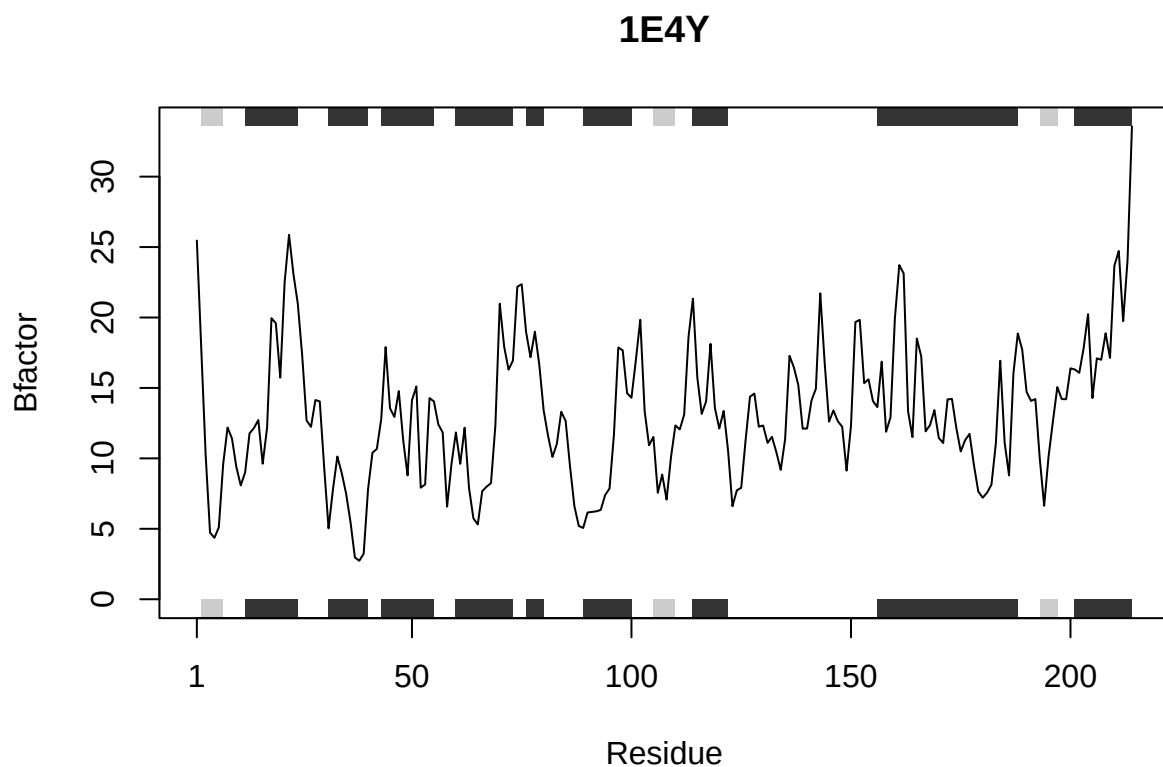


```
## Note: Accessing on-line PDB file
```

```
## Warning in get.pdb(file, path = tempdir(), verbose = FALSE):
```

```
## /var/folders/2v/sltcs3bj6w50qp7pg2bk1kl80000gn/T//RtmpG6auwD/1E4Y.pdb exists.
```

```
## Skipping download
```



This final function iteration is taking a vector of PDB Identifications as inputs and is analyzing them. It is pulling which chain, atom type (most likely Alpha Carbon), and Beta-factor the user requests. It produces a Beta-Factor plot similar to the original code.